IN-DEPTH PROJECT REFERENCE

Delayed Gradients in CPU-Based Neural Network Training

Theory, implementation strategy, experiment design, and interpretation of the project result files

PROJECT QUESTION

How much synchronization waiting can CPU workers remove while keeping training useful?

Executive summary

The project is a CPU C++ prototype of a small MNIST classifier. It compares a sequential implementation, a one-batch synchronous worker implementation, and a two-worker bounded delayed-gradient implementation. The delayed mode allows the output worker to get ahead while the hidden-layer owner accepts only gradients whose model-version age is within a chosen bound.

- Network: 784 input values -> 32 ReLU hidden units -> 10 output logits.

- Experiments: 5,000 training images per epoch, 1,000 test images, five epochs, learning rate 0.01, seed 12345.

- The main variable is tau_max in {0, 1, 2, 4, 8, 16}; delayed runs use up to 8 batches in flight.

- The key result is a measured speed-quality tradeoff, not proof of a universal CPU speedup.

How to use this guide

Read Sections 1-4 to understand the algorithm and code structure. Use Sections 5-8 to interpret the CSVs and workbook. Section 9 identifies data caveats and filenames to check before sharing results.

1. Theory: the learning task

MNIST contains grayscale handwritten digit images labeled 0 through 9. A 28 by 28 image is flattened into 784 values and normalized from byte intensity [0,255] to floating-point [0,1]. The model is a multilayer perceptron with one hidden dense layer.

Forward pass

```
a0 = W0 x + b0; h = ReLU(a0)
z = W1 h + b1; p = softmax(z)
```

Here x is the pixel vector, h is the hidden activation, z is the ten-class logit vector, and p gives class probabilities. ReLU returns max(0, value). The predicted digit is the index of the largest logit/probability.

Loss and backpropagation

For the correct class y, cross-entropy is L = -log(p_y). The output-logit derivative for softmax plus cross-entropy is p - one_hot(y). Backpropagation sends this error signal through W1 to the hidden layer, then applies the ReLU derivative. Each layer's weight and bias gradients come from its input and its local error signal.

Gradient descent updates a parameter vector theta by theta_new = theta_old - learning_rate * gradient. Both implementations use learning rate 0.01. The sequential version computes both layers' gradients from the same forward pass before applying either update; each owner-worker modifies only its own layer.

Why threading introduces waiting

Layer dependencies constrain a neural-network pipeline. Worker 1 cannot compute output logits until Worker 0 sends hidden activations. Worker 0 cannot finish hidden-layer backpropagation until Worker 1 sends the derivative with respect to the hidden activations. With strict synchronization, a worker blocks until its required message is ready. More threads do not automatically mean more useful computation.

Project theory source

The supplied implementation guide describes delayed gradients as real gradients produced from an older version of model weights. The project operationalizes the idea with version numbers and a staleness policy; it does not invent or predict gradients.

2. What gradient age means

Every layer owner has a monotonically increasing version. When Worker 0 produces hidden activations for a batch, the activation message records Worker 0's version. Worker 1 later returns a gradient tagged with that origin version. When Worker 0 receives it, it compares the origin with its current version.

```
age = current_hidden_version - gradient_origin_version
if age <= tau_max: apply the hidden-layer update
otherwise: discard the hidden-layer gradient
```

Example: if the gradient came from hidden version 101 and Worker 0 is now at version 105, the age is 4. With tau_max=4 it is accepted; with tau_max=2 it is rejected. Age is a count of model-version updates, not milliseconds.

Why the bound matters

A smaller bound keeps updates closer to current weights, but it can reject more gradients when work overlaps. A larger bound accepts more gradients and can reduce waiting or buffering pressure, but allows updates based on older weights. The relationship is empirical: a higher tau_max does not guarantee faster convergence or better accuracy.

A special distinction: tau_max=0 is not always synchronous

The synchronous reference uses tau_max=0 and max_in_flight=1. A delayed run with tau_max=0 and max_in_flight=8 still allows messages to overlap; gradients that arrive one version old are discarded. The two configurations share the same age limit but have different queue concurrency and learning behavior.

Output-layer updates continue even when a hidden-layer gradient is discarded. Therefore completed examples, output-layer updates, accepted hidden updates, and completed-example throughput are different counts. Interpret them together.

Initialization

DenseLayer initializes weights from a seeded normal distribution. The scale is sqrt(2/fan_in) for the ReLU hidden layer and sqrt(1/fan_in) for the linear output layer; biases start at zero. The hidden layer uses seed 12345 and the output layer seed 12346.

3. Implementation strategy

The implementation uses fixed C++17 threads, owner-only layer updates, bounded blocking queues, mutexes, and condition variables. Each DenseLayer is written by only its owner thread, which avoids concurrent unsynchronized writes to the same parameter vectors.

Stage	Message / action	Owner and purpose
1	Coordinator pushes TrainingMsg (batch ID, label, pixels)	Worker 0 receives input and caches it by batch ID.
2	Worker 0 computes hidden activation; pushes ActivationMsg	Message carries batch ID and source hidden-layer version.
3	Worker 1 computes logits, cross-entropy, gradients	Worker 1 owns output layer; computes hidden-activation gradient.
4	Worker 1 applies output update; pushes GradientMsg	Gradient contains batch ID, origin version, producer version, loss, and vector.
5	Worker 0 checks age; applies or discards	If accepted, uses cached input for hidden-layer backprop and updates its layer.
6	Worker 0 pushes BatchResultMsg	Coordinator tracks completion, loss, acceptance, and gradient age.

Queues and backpressure

The queues have finite capacity. push() blocks while a queue is full; pop() blocks while it is empty. These condition-variable waits provide producer-consumer synchronization and prevent unbounded message accumulation. The coordinator enforces its in-flight window using submitted_count - completed_count < max_in_flight.

Why the hidden worker has a mixed inbox

The coordinator sends TrainingMsg objects to Worker 0, while Worker 1 sends GradientMsg objects back to that same owner. HiddenWorkMsg is a variant of those two types. Worker 0 processes them serially, so one thread remains the sole writer of hidden-layer weights.

Shutdown

At the end of an epoch all batch results have returned, so the coordinator can evaluate the model without workers changing its weights. At final completion it closes the work queue and joins the worker threads. Closing queues wakes threads blocked on push or pop.

4. Experiment design and metrics

The experiment ladder separates the cost of threads from the effect of delayed gradients.

Mode	Configuration	What it isolates
Sequential	One thread updates both layers	Reference training time and learning curve.
Synchronous workers	2 workers; tau_max=0; max_in_flight=1	Worker/queue overhead under a one-batch handshake.
Bounded delayed	2 workers; max_in_flight=8; tau_max swept	Effect of bounded version staleness and overlap.

Controlled settings

- The same two-layer architecture, training subset size, test subset size, learning rate, and random seed are used for the reported five-epoch comparisons.

- Training uses 5,000 selected examples per epoch; test metrics use a fixed selected subset of 1,000 images.

- The worker program shuffles the selected training subset each epoch. It waits for all epoch results before evaluating the checkpoint.

- Epoch training time excludes test evaluation. It includes training coordination and queue/worker pipeline time; worker thread startup occurs before epoch timing begins.

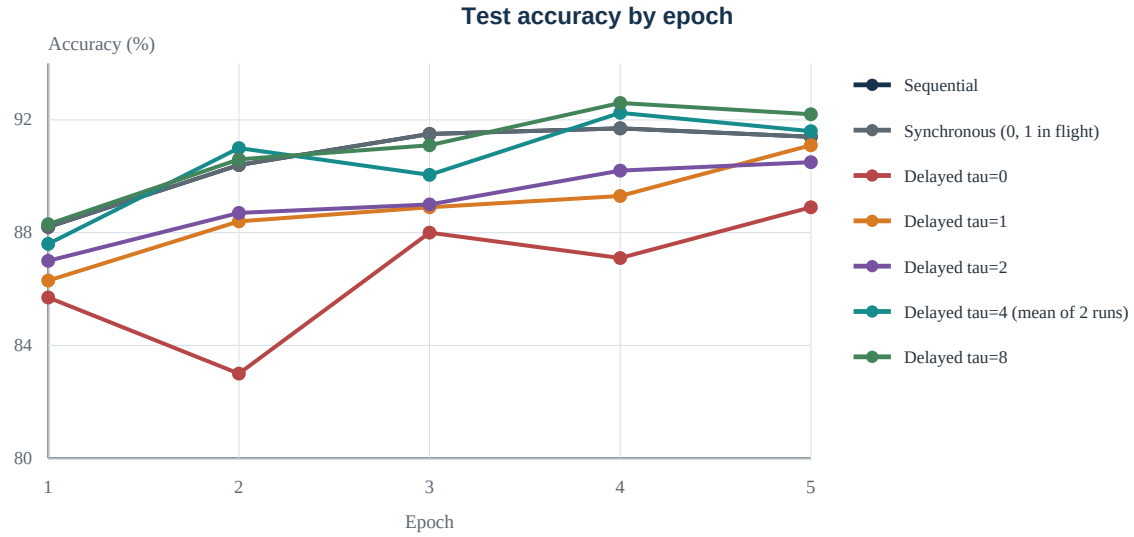
- The primary five-epoch sweep records tau_max=0,1,2,4,8. A prior one-epoch repeated sweep also included 16.

Metric	Interpretation
Training loss	Average pre-update cross-entropy for training examples processed that epoch, including examples whose hidden gradient was later discarded.
Test loss / accuracy	Average cross-entropy and fraction correct on the selected 1,000-image test subset at an epoch checkpoint.
Time to 90%	Elapsed training time at the first epoch checkpoint with test accuracy >= 90%; not the exact within-epoch crossing time.
Accepted update %	Hidden-layer gradients applied divided by completed examples. Output-layer updates still occur for each processed activation.
Gradient age	Difference between Worker 0's current version and the activation's source version when the gradient returns.
Worker wait	Accumulated time inside blocking input-queue pop calls; not CPU utilization and not a complete run/wait state timeline.

5. Reading epoch_curves.csv and the workbook

epoch_curves.csv stores one row per epoch checkpoint. Its columns identify configuration, tau_max, run number, epoch, average training loss, test loss, test accuracy, and cumulative training seconds. There are 45 checkpoint rows in the saved version: five epochs for nine runs.

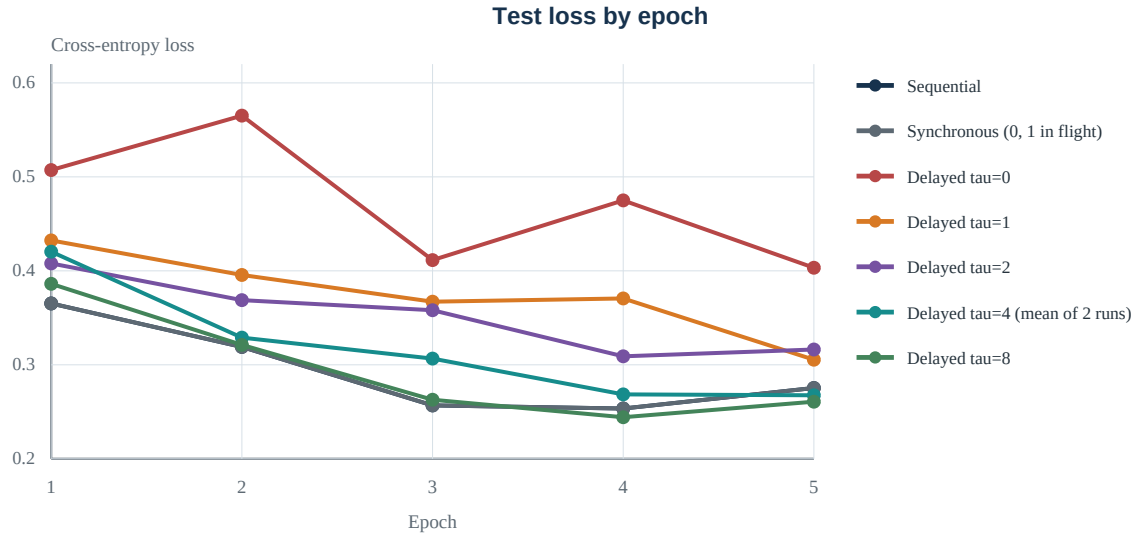
For the overview charts below, duplicate sequential runs and the two tau_max=4 runs are averaged at each epoch. The original per-run points remain in the CSV and should be available when discussing variability.



Interpretation: all tested settings learn beyond the 7.2% untrained baseline, but strict stale-gradient rejection slows or limits learning in these runs. The delayed tau=0 curve fluctuates and finishes at 88.9%. tau=1 and tau=2 reach the 90% region later. tau=4 and tau=8 reach it by epoch 2 in recorded runs. Test accuracy can dip after an earlier higher checkpoint; training loss falling does not force test accuracy to rise monotonically.

6. Test-loss curve and its meaning

Test loss measures the model's confidence-sensitive error on held-out examples. A wrong prediction made with high confidence incurs more loss than a hesitant wrong prediction. Accuracy only counts whether the top-scoring class is correct, so the two metrics can move differently.



Observed pattern

Most runs reduce test loss substantially from the untrained value of about 2.367. Curves are not strictly monotonic: for example, some configurations have a lower loss at an earlier checkpoint than at epoch 5. This can reflect generalization variation on the selected test subset, asynchronous update ordering, and the small training subset. Do not present only the final point if the curve contains meaningful dips or rebounds.

Workbook

The project results folder currently contains `curves.xlsx`. Inspection found worksheets named `epoch_curves`, `test accuracy by epochs`, `test loss by epochs`, and `time to 90 vs max gradient`. The actual workbook name differs from `epoch_curves.xlsx`; use the existing `curves.xlsx` unless the team renames it deliberately.

7. Reading epoch_comparison.csv

This file compresses each five-epoch run to one row: run ID, mode, tau_max, in-flight limit, hidden-update acceptance percentage, final test accuracy/loss, total training time, first epoch at or above 90%, and time at that checkpoint.

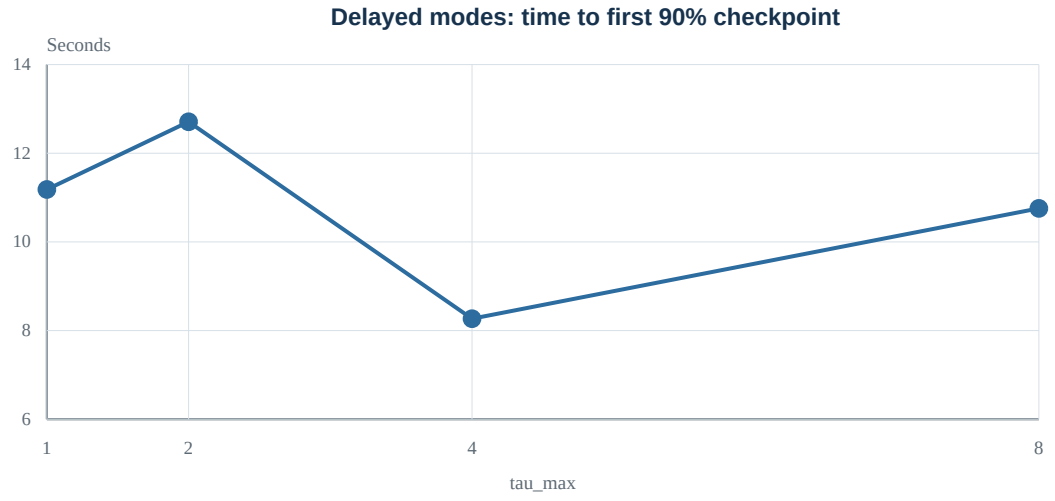
Setting	Acceptance	Final accuracy	Time	Time to 90%
Sequential run 1	100.00%	91.4%	26.92 s	10.88 s
Sequential run 2	100.00%	91.4%	29.63 s	11.38 s
Synchronous (0,1)	100.00%	91.4%	28.61 s	11.54 s
Delayed tau=0 run 1	12.50%	88.9%	10.00 s	Not reached
Delayed tau=1 run 1	27.168%	91.1%	11.18 s	11.18 s
Delayed tau=2 run 1	47.968%	90.5%	15.76 s	12.71 s
Delayed tau=4 run 1	69.160%	92.1%	19.91 s	8.02 s
Delayed tau=4 run 2	71.400%	91.1%	21.14 s	8.52 s
Delayed tau=8 run 1	100.00%	92.2%	26.58 s	10.76 s

What the table supports

The sequential runs finished at 91.4% in 26.92 and 29.63 seconds. The single synchronous row in this CSV finished at 91.4% in 28.61 seconds. Delayed tau=4 has two rows at 92.1%/19.91 seconds and 91.1%/21.14 seconds; both crossed 90% at epoch 2. tau=0 with 8 in flight was fastest at 10.00 seconds but finished at 88.9%, below the 90% target. tau=8 accepted all hidden gradients and finished at 92.2%, but took 26.58 seconds.

The result is not a simple monotonic curve. In these runs tau=4 offered a useful compromise between accepted updates, time-to-target, and final accuracy. That is an empirical observation from this machine and subset, not a guaranteed optimal bound.

Time-to-target summary chart



tau=0 is not plotted because that run did not reach 90% within five epochs. tau=4 is averaged over the two runs included in epoch_comparison.csv; the additional instrumented run is in worker_timing_runs.txt and is not yet included in this CSV.

8. Worker timing and systems interpretation

The current worker metrics measure time inside blocking input queue pops and a broad active path that includes processing plus queue pushes. They show pipeline imbalance, but they are not a CPU profiler. Per-thread totals overlap, and a worker's cumulative wait can exceed wall-clock duration because timing intervals overlap startup/shutdown and other worker activity.

Configuration (from numeric settings)	Wall time	W0 active path	W0 input wait	W1 active path	W1 input wait
Delayed tau=4, in flight=8	18.727 s	18.690 s	1.102 s	0.817 s	18.966 s
Synchronous tau=0, in flight=1	28.951 s	27.146 s	2.851 s	0.586 s	29.421 s

Interpretation

Worker 0 performs the 784-to-32 forward calculation and hidden backpropagation. Its active path dominates both configurations. Worker 1's small 32-to-10 layer has little compute and spends much of the measured run waiting for activations. The delayed configuration's absolute Worker 1 input wait is lower in this sample largely alongside its shorter wall time. This points to an imbalanced two-stage pipeline: allowing stale gradients does not make the output stage computationally heavy or remove the hidden-stage bottleneck.

The timing artifact has a label problem

In `results/worker_timing_runs.txt`, the section heading `Synchronous (0 1 5)` is followed by output settings tau_max=4 and max_in_flight=8. The section heading `Delayed (4 8 5)` is followed by tau_max=0 and max_in_flight=1. The headings appear swapped. The table above identifies each run from the program's numeric settings, not from those headings. Correct the headings before sharing that text file.

This project does not report queue-depth distributions, memory use, CPU utilization, or a per-worker RUN/WAIT timeline. Do not claim those metrics were measured. The worker pop-wait measurement is useful evidence, but it is narrower than total idle time.

9. Overall findings, limitations, and claims

What the experiments suggest

- The synchronous worker version behaves like a strict one-batch handshake. It provides an OS-thread/queue baseline but was not faster than sequential in the recorded five-epoch timings.

- With tau_max=0 and 8 batches in flight, overlap increases completed-example rate, but many hidden gradients are discarded and final accuracy is lower. A high examples-per-second number alone is therefore misleading.

- tau_max=1 and 2 accepted more hidden updates than tau=0/8, but their time-to-90 and final accuracy differed. The sweep is non-monotonic.

- tau_max=4 reached the 90% checkpoint quickly in repeated runs and had final accuracies around 91-92%. This is the strongest compromise in the recorded five-epoch comparison, with run-to-run variation.

- tau_max=8 accepted all hidden gradients in its run and reached 92.2%, but its elapsed time was close to sequential/synchronous time.

Limitations to say aloud

- Only a subset of MNIST is used: 5,000 train examples per epoch and 1,000 test examples.

- Most five-epoch settings have one run; sequential and tau=4 have more. Do not infer statistical significance.

- The CPU model, compiler build, background load, and OS scheduler affect timings. The exact hardware/compiler details should be added if known.

- Epoch checkpoint time to 90% is a discrete checkpoint measurement, not exact time when accuracy crossed 90% inside an epoch.

- Completed examples include samples whose hidden-layer gradients were discarded. Report acceptance rate and accepted-update rate as context.

- The one-hidden-layer model is a small systems prototype, not a state-of-the-art MNIST model.

A defensible conclusion

In these experiments, bounded delayed-gradient execution changed both pipeline waiting and learning outcomes. A delay limit of 4 gave a useful observed speed-quality compromise, while the strictest asynchronous setting showed why throughput must be considered alongside accepted updates and time-to-target.

10. How to use the three results artifacts

Artifact	What it contains	How the teammate should use it
results/epoch_comparison.csv	One row per five-epoch run; final metrics, accepted updates, total time, epoch/time to 90%.	Use for summary comparison and time-to-target chart. Preserve run IDs; avoid merging repeated runs without stating the averaging method.
results/epoch_curves.csv	One row per epoch checkpoint for each recorded run; training loss, test loss, accuracy, cumulative training seconds.	Use for accuracy/loss learning curves. Run/config fields identify separate traces.
results/curves.xlsx	Observed workbook with data and sheets for accuracy, loss, and time-to-target charts.	Use as the chart presentation file. It is currently named curves.xlsx, not epoch_curves.xlsx.

Artifact consistency checks before sharing

- The actual chart workbook found in the results directory is `curves.xlsx`; no file named `epoch_curves.xlsx` was found there. Rename only if the team wants to standardize the name, then update references.

- The worker timing text has reversed section headings, as explained in Section 8. Fix those labels before sharing or cite the numeric configuration fields.

- The latest instrumented tau=4 run and latest synchronous run are in worker_timing_runs.txt but are not represented as new rows in epoch_comparison.csv or epoch_curves.csv. Keep this distinction clear in charts and tables.

- The README found in the project folder stops partway through the sequential build command. Complete it before sending it as the teammate's project overview.

Other useful handoff files

- README.md after it is completed; results/demo_script.md; the original Approach_1_Delayed_Gradients_Implementation_Guide.pdf; and the source folder src/.

- The MNIST data folder is unnecessary for building slides from saved results. Include it only if the teammate must rerun the experiment.

- Compiled .exe files are not a substitute for source and do not need to be included for slide preparation.

Technical glossary

Term	Meaning in this project
Owner worker	The only thread allowed to update its DenseLayer weights/biases.
Version	Counter incremented when the layer owner applies an update.
Gradient age	Current hidden-layer version minus the activation's source version.
tau_max	Largest hidden-gradient age accepted by Worker 0.
In flight	Submitted batches not yet returned as completed BatchResult messages.
Backpressure	A producer blocks because a bounded queue is full.
Time to target	Elapsed training time at first epoch checkpoint reaching a target accuracy.